EXERCISE 2

Demagnetizing Field via FFT

Task 1: Demagnetizing field in 1D via FFT convolution

This report develops and validates the one-dimensional demagnetizing field calculation for a row of unit-sized cells. The direct O(N²) method is compared with a zero-padded FFT convolution of O(N log N) complexity.

$$\vec{h}(\vec{r}) = \frac{1}{4\pi r^3}\left[3(\vec{m}\cdot\hat{r})\hat{r} - \vec{m}\right]$$

All fields below use the point-dipole model from the exercise sheet. The self-interaction is treated separately because the point kernel is singular at the source position.

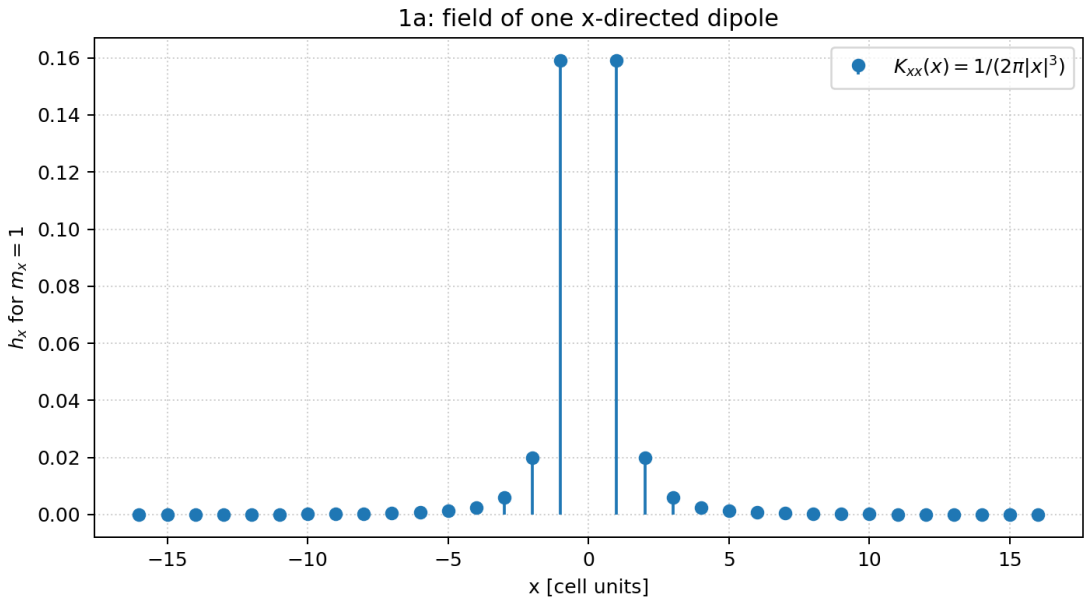
1a - Single dipole and one-dimensional kernel

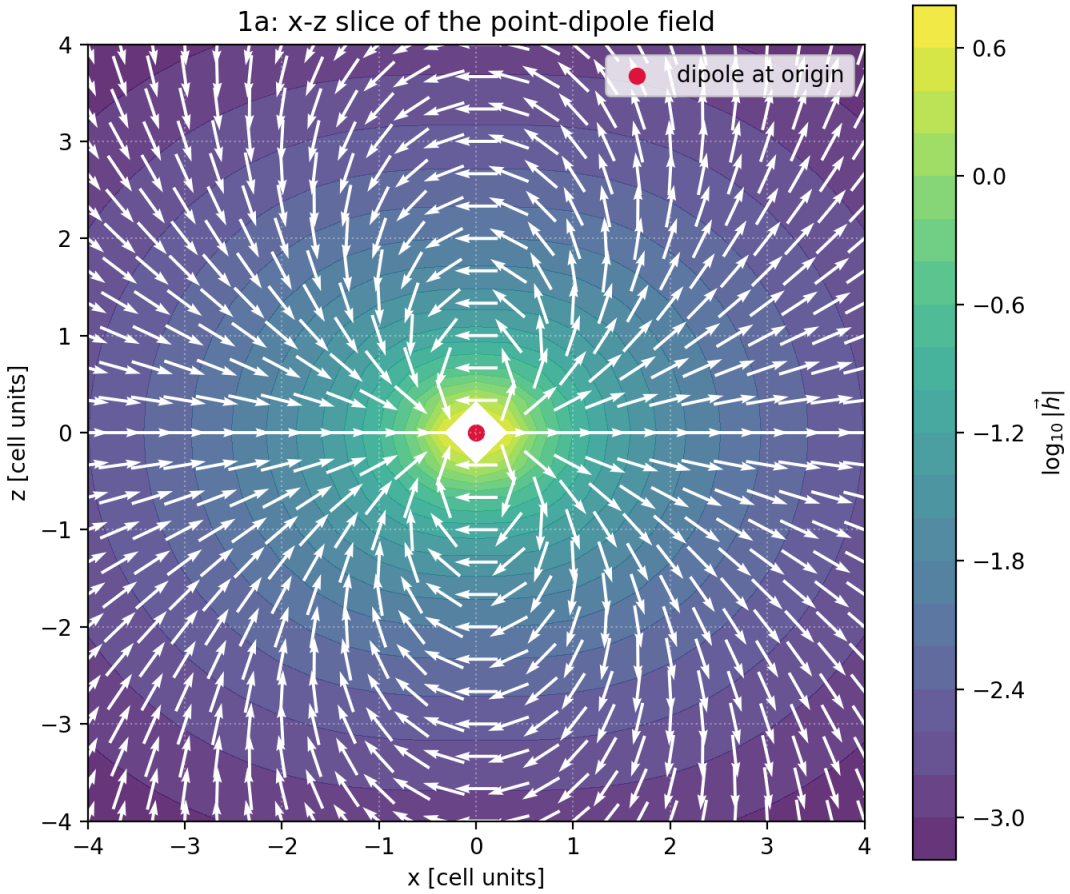
For an x-directed dipole evaluated on the x-axis, the unit vector is parallel or antiparallel to the x-axis. Substitution into the dipole formula gives an even, positive x-component on both sides of the dipole:

$$\vec{m} = m_x\hat{x},\ \vec{r} = x\hat{x} \quad \Longrightarrow \quad h_x(x) = K_{xx}(x)m_x = \frac{m_x}{2\pi|x|^3}, \quad x \neq 0$$

Thus the field magnitude decays as |x|^-3. The point-dipole expression is undefined at x = 0. In the implementation the self term is excluded by setting K_xx(0) = 0. This means that a cell does not act on itself. A nonzero self-term would require a finite-cell, volume-averaged demagnetizing tensor rather than the point-dipole kernel.

```
def dipole_kernel_1d(size):
    distances = np.arange(size, dtype=float)
    kernel = np.zeros(size, dtype=float)
    nonzero = distances != 0
    kernel[nonzero] = 1.0 / (2.0 * np.pi * distances[nonzero]**3)
    return kernel
```





The second figure shows the full vector field in the x-z plane. Arrow directions are normalized for visibility; the color represents log10 of the field magnitude. The origin is omitted from the analytic field because it is singular.

1b - Direct summation

$$h_i = \sum_{j \neq i} K_{xx}(i-j) m_j = \sum_{j \neq i} \frac{m_j}{2\pi |i-j|^3}$$

The direct method evaluates every source cell for every evaluation cell. For a uniformly magnetized segment, the field is smaller at the two ends because one side has no magnetized source cells. In the middle, contributions arrive from both sides, so the field is approximately symmetric and nearly constant away from the boundaries. The long-ranged kernel still produces visible edge effects.

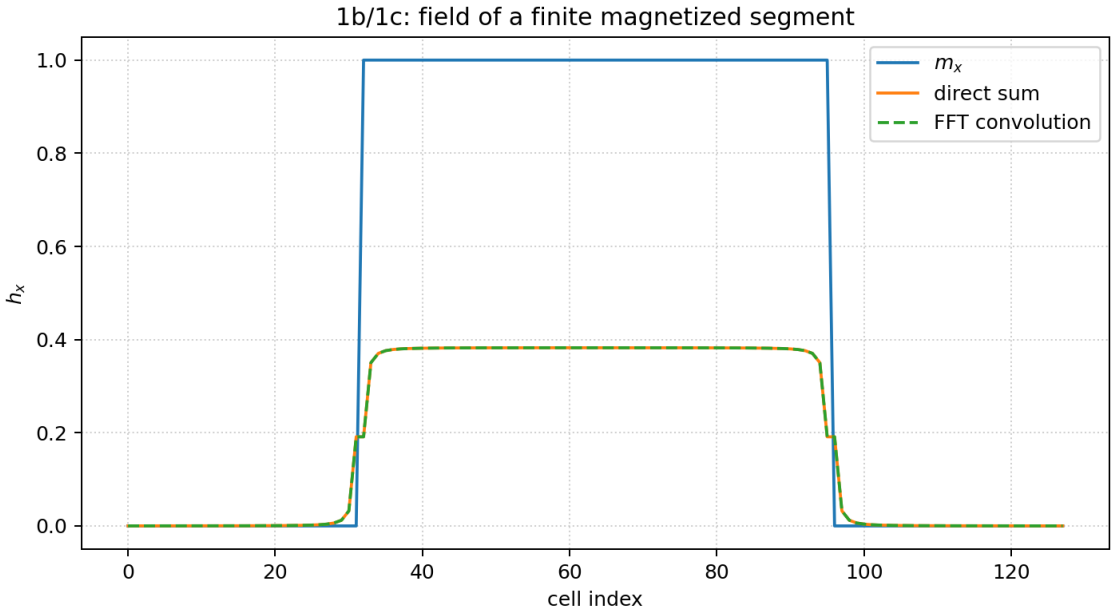
```
indices = np.arange(values.size)
distances = np.abs(indices[:, None] - indices[None, :])
kernel = np.zeros_like(distances, dtype=float)
nonzero = distances != 0
kernel[nonzero] = 1.0 / (2.0 * np.pi * distances[nonzero]**3)
field = kernel @ values
```

1c - Linear convolution with the FFT

$$\widehat{h}_k = \widehat{K}_k \widehat{m}_k$$

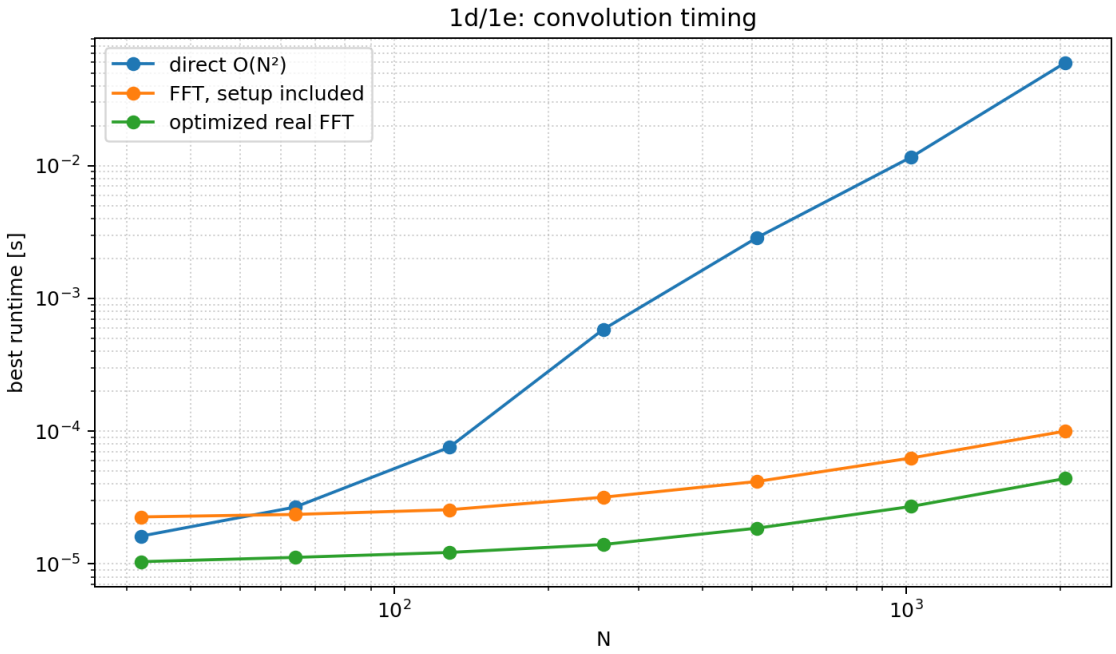
The discrete Fourier transform produces a circular convolution. To obtain the desired linear convolution, both the magnetization and the displacement kernel are embedded in arrays of length L >= 2N - 1. The kernel is arranged as K[0] = 0, K[d] = 1/(2πd³) for d = 1,...,N-1, and K[L-d] = K[d]. The second half stores the negative displacements at their wrapped FFT indices. Zero padding prevents contributions from wrapping around into the physical output.

```
kernel = np.zeros(fft_size)
kernel[:size] = dipole_kernel_1d(size)
kernel[-(size - 1):] = kernel[1:size][::-1]
field = np.fft.irfft(
    np.fft.rfft(padded_magnetization) * np.fft.rfft(kernel),
    n=fft_size,
)[:size]
```



1d - Scaling and benchmark

The direct algorithm evaluates O(N²) pair interactions. The FFT algorithm uses O(L log L) operations, with L approximately 2N. Consequently the FFT method becomes increasingly faster as N grows. N does not have to be a power of two: NumPy supports arbitrary transform lengths, although lengths with small prime factors are generally faster. The implementation selects a 2/3/5-smooth length at least 2N - 1.



The benchmark table below is generated from the same implementation and records the fastest of three timed calls for each size. The numerical errors remain at floating-point round-off level.

N	FFT length	direct [s]	FFT [s]	optimized [s]	max. error
32	64	1.51e-5	2.22e-5	1.00e-5	1.7e-16
128	256	7.58e-5	2.52e-5	1.22e-5	3.3e-16
512	1024	2.81e-3	4.26e-5	1.85e-5	2.8e-16
1024	2048	1.13e-2	6.32e-5	2.70e-5	4.4e-16
2048	4096	4.95e-2	1.02e-4	4.35e-5	5.6e-16

1e - Further FFT optimization

The optimized implementation uses real-input transforms rfft/irfft, computes the symmetric kernel spectrum once, reuses that spectrum for repeated evaluations with the same N, and chooses an FFT-friendly length. These changes reduce memory use and avoid redundant work without changing the linear-convolution result. For repeated calculations, kernel-spectrum reuse is the most important algorithmic optimization. FFT planning would be an additional optimization in libraries that expose reusable transform plans.

```
fft_size = next_fast_length(2 * size - 1)
kernel_spectrum = _fft_kernel_spectrum(size, fft_size)
field = optimized_fft_convolution(
    magnetization, kernel_spectrum, fft_size
)
```

Conclusion: direct summation is useful for validation, while the zero-padded real FFT provides the expected O(N log N) scaling. Agreement between both methods confirms the kernel arrangement and implementation.